ManpowerGroup - Weekend Take-Home

Task: Build a Streamlit app that lets a user upload **any PDF** and chat with it.
Flow: PDF upload → chunking → embeddings → store in vector DB → retrieve → Q&A with citations

What to Build (Minimum Requirements)

1. **Streamlit UI**
    - Upload **any** PDF (single file is fine).
    - A chat interface with **persistent conversation history** in the app (left pane or within chat).
    - Show **sources/citations** (e.g., page # or snippet) for each answer.
2. **Processing Pipeline**
    - **Chunking:** sensible PDF text extraction; state your chunking strategy.
    - **Embeddings:** any model (OpenAI, HF, BGE, etc.). Note model name and dimensions in README.
    - **Vector Store:** any vector database (Chroma, FAISS, LanceDB, Qdrant, etc) of your choice.
    - **Retrieval:** mention the retrieval strategy and the reason.
3. **Q&A (RAG)**
    - Build a prompt that **uses retrieved chunks** as context.
    - The answer must be **grounded** and include **citations** (page/line or snippet).
    - Handle **follow-up questions** (use chat history in the prompt or RAG pipeline).
4. **Run It Locally**
    - streamlit run app.py (or similar) should boot the app after pip install -r requirements.txt.

Nice-to-Haves (Optional, bonus points)

- **Basic eval**: 5–10 test questions + an accuracy table (manual judgment is okay).
- **Modular Code:** code files properly structured into modules

You May Use Any AI Tools: But Share Prompt Logs

- You can use **Cursor, ChatGPT, Claude, Gemini, Copilot, etc.** However, you **must include your conversation history/prompt logs** (screenshots or exported text) used to build/debug the app so we can assess your **prompting and iteration**.
- You can use any APIs for embedding & LLMs (OpenAI, OpenRouter, TogetherAI, etc)

Deliverables (one repo + one short doc)

1. **Git repo**
    - Save all your code files in github
    - requirements.txt
    - Local persistence path for vector DB (e.g., ./storage/)
2. **README.md (max 1–2 pages)**
    - Setup steps (including environment variables structure).
    - Models used (embedding + LLM) API, chunking strategy, vector DB choice.
    - How conversation history is maintained in the app.
    - Known limitations.
3. **Prompt Logs (mandatory)**
    - Screenshots or exports of your **AI assistant chats** used during development.
4. **Demo Evidence**
    - A short screen recording (≤3 min) or 5 screenshots showing: upload → 5+ Q&A turns → citations.
    - Include the **PDF (s)** you used for the demo (public docs only) or a link.

Acceptance Test (what we'll do)

- Run pip install -r requirements.txt and streamlit run app.py.
- Upload a **random public PDF** (not included in your repo).
- Ask 5 questions; expect:
    - **Grounded answers** (drawn from retrieved text)
    - **Citations** (page/snippet)
    - **Follow-ups** leveraging chat history
    - No crashes